

Kỹ Thuật Lập Trình Python

Python và Phát triển Web django



- 
1. Giới thiệu lộ trình lập trình web
 2. Giới thiệu Model – View – Controller (MVC)
 3. Phát triển web bằng django framework
 4. Giới thiệu Model – View – Template (MVT)
 5. Ứng dụng minh họa
- 

Lộ trình

- Có nhiều tiếp cận nhưng có thể tham khảo lộ trình **cơ bản** sau:
 1. HTML, CSS
 2. Javascript, JQuery
 3. Frontend framework - CSS framework
 4. Python
 5. Web framework: Flask, Django...
 6. Database: MySQL, PostgreSQL và SQLite...

Python framework hỗ trợ lập trình web

- Django
- Flask



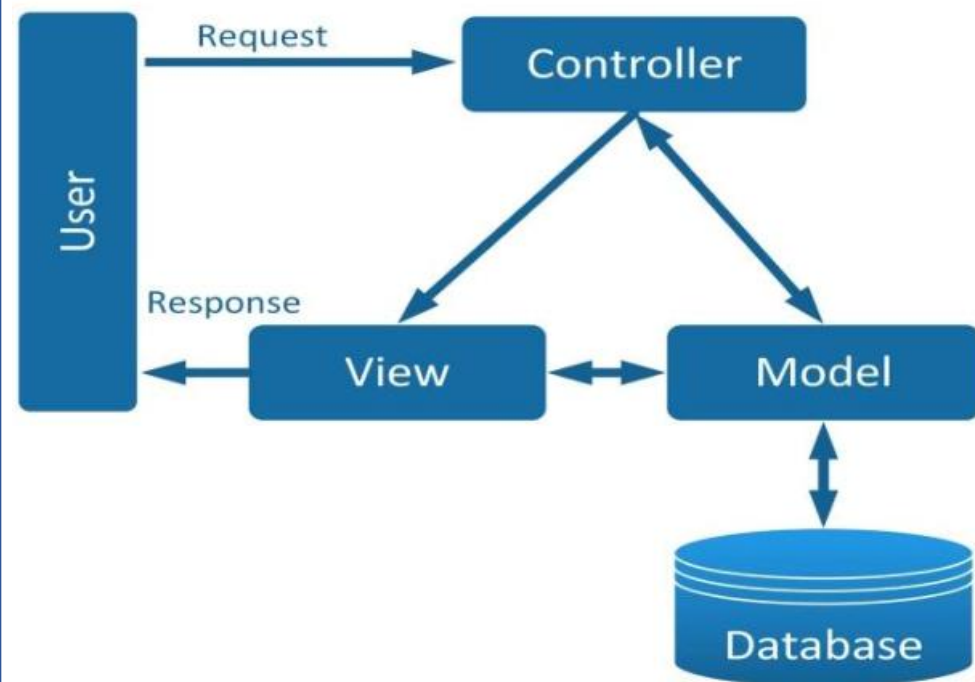
Tham khảo: <https://www.activestate.com/blog/the-top-10-python-frameworks-for-web-development/>

Model – View – Controller (MVC)

Giới thiệu

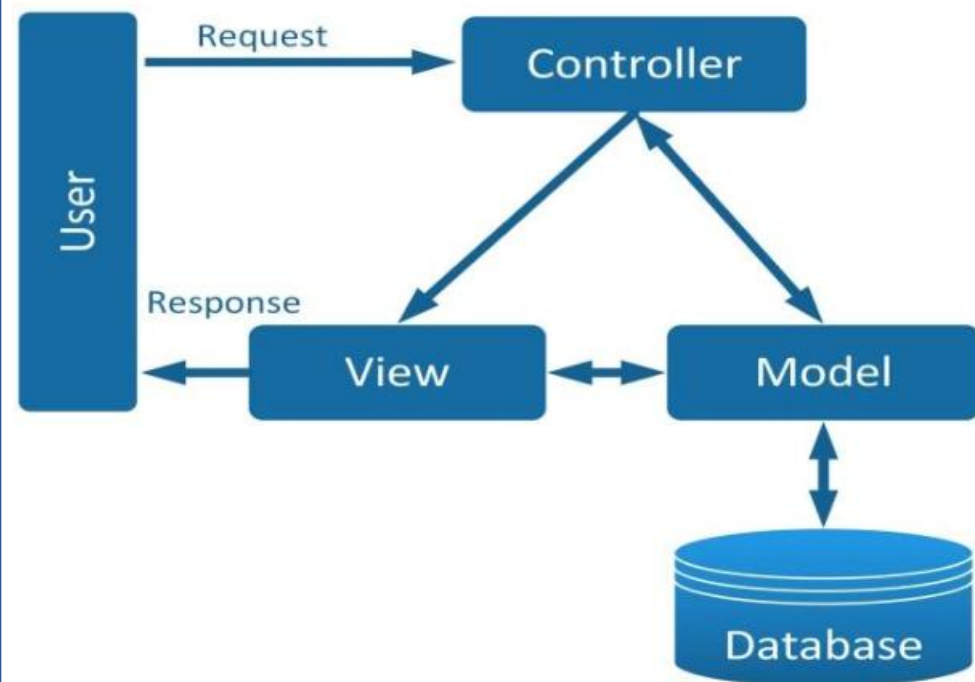
- MVC là một mô hình thiết kế web phổ biến hiện nay để thiết kế website, web app, ứng dụng di động.
- Phù hợp cho nhiều ngôn ngữ như là JavaScript, PHP, Ruby, Python (MVT)...

Model – View – Controller (MVC)



- **M** là **Model**: một cấu trúc dữ liệu, chuẩn bị dữ liệu để cung cấp cho Controller.
 - Model chứa các logic, tương tác với dữ liệu, lưu trữ tất cả các CSDL, và là cầu nối giữa View và Controller.
 - Model gồm những thao tác xử lý kết nối CSDL, cho phép xem, xử lý dữ liệu, truy xuất,...
- **V** là **View**: là nơi hiển thị dữ liệu cho User (có thể gọi là giao diện người dùng)
 - User có thể tương tác lên View.
 - View thường là nơi mà thành phần HTML được tạo ra.
- **C** là **Controller**:
 - Nhận lệnh từ người dùng, gửi lệnh đến **Model** để lấy hoặc cập nhật dữ liệu,
 - Truyền lệnh đến **View** để cập nhật giao diện hiển thị cho đúng với dữ liệu đã cập nhật.

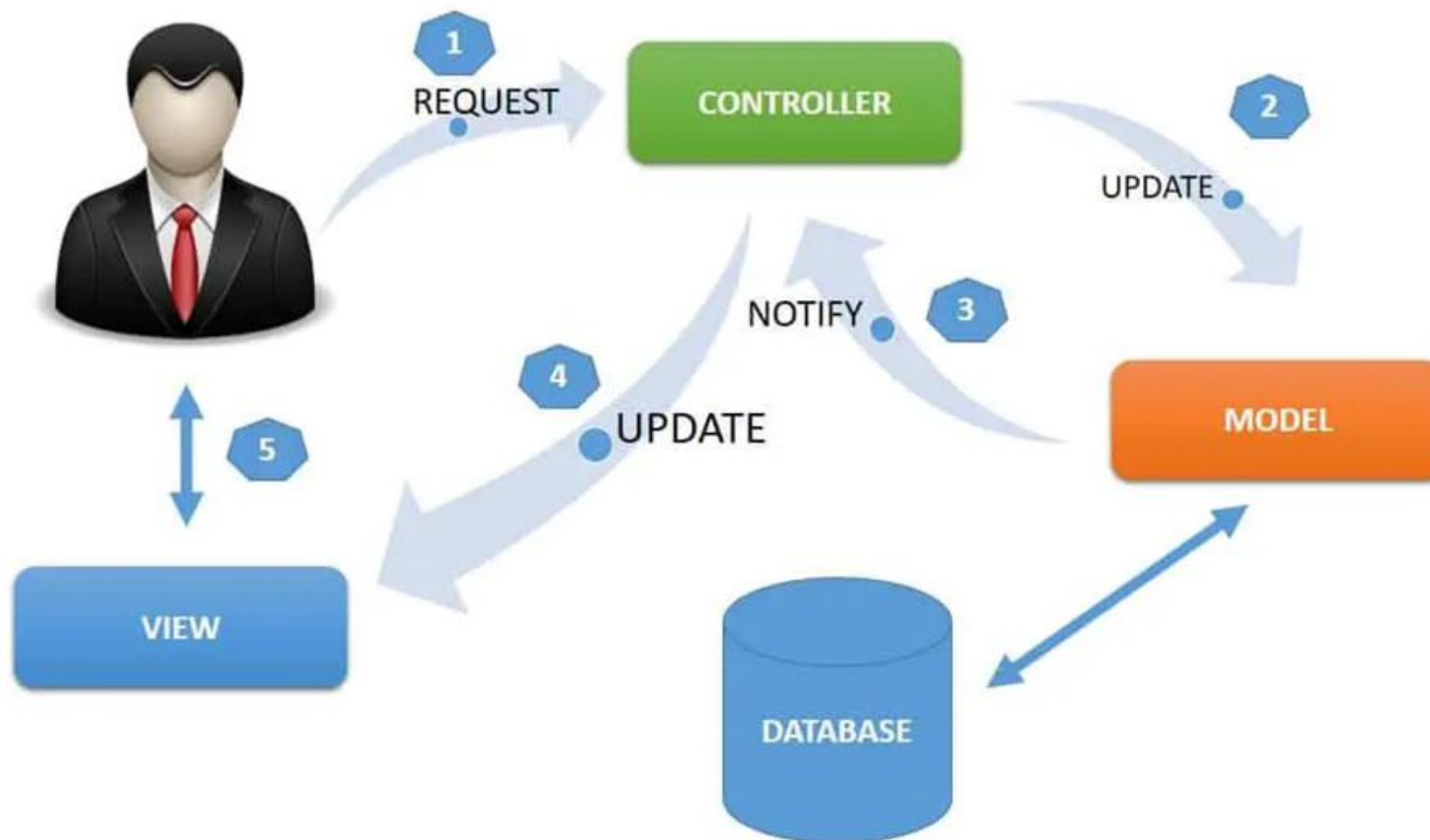
Model – View – Controller (MVC)



- **User** tạo ra yêu cầu thông qua đường dẫn.
- **Controller** nhận yêu cầu và đưa ra một mệnh lệnh để xử lý yêu cầu.
 - Nếu lệnh thực thi đối với **View** thì cập nhật lại giao diện hiển thị;
 - Nếu lệnh thực thi đối với **Model** thì xử lý theo cầu của người dùng.
- **Model** thực thi phần xử lý, tương tác CSDL rồi gửi trả lại phản hồi theo hướng dẫn từ **Controller**
- **Controller** truyền dữ liệu này ra **View** và cập nhật lại giao diện cho User.
- Mọi yêu cầu đều phải đi qua **Controller** trước khi chuyển hóa thành lệnh thực thi cho **View** hay **Model**. => **Controller** được xem là bộ não.

Model – View – Controller (MVC)

Tương tự



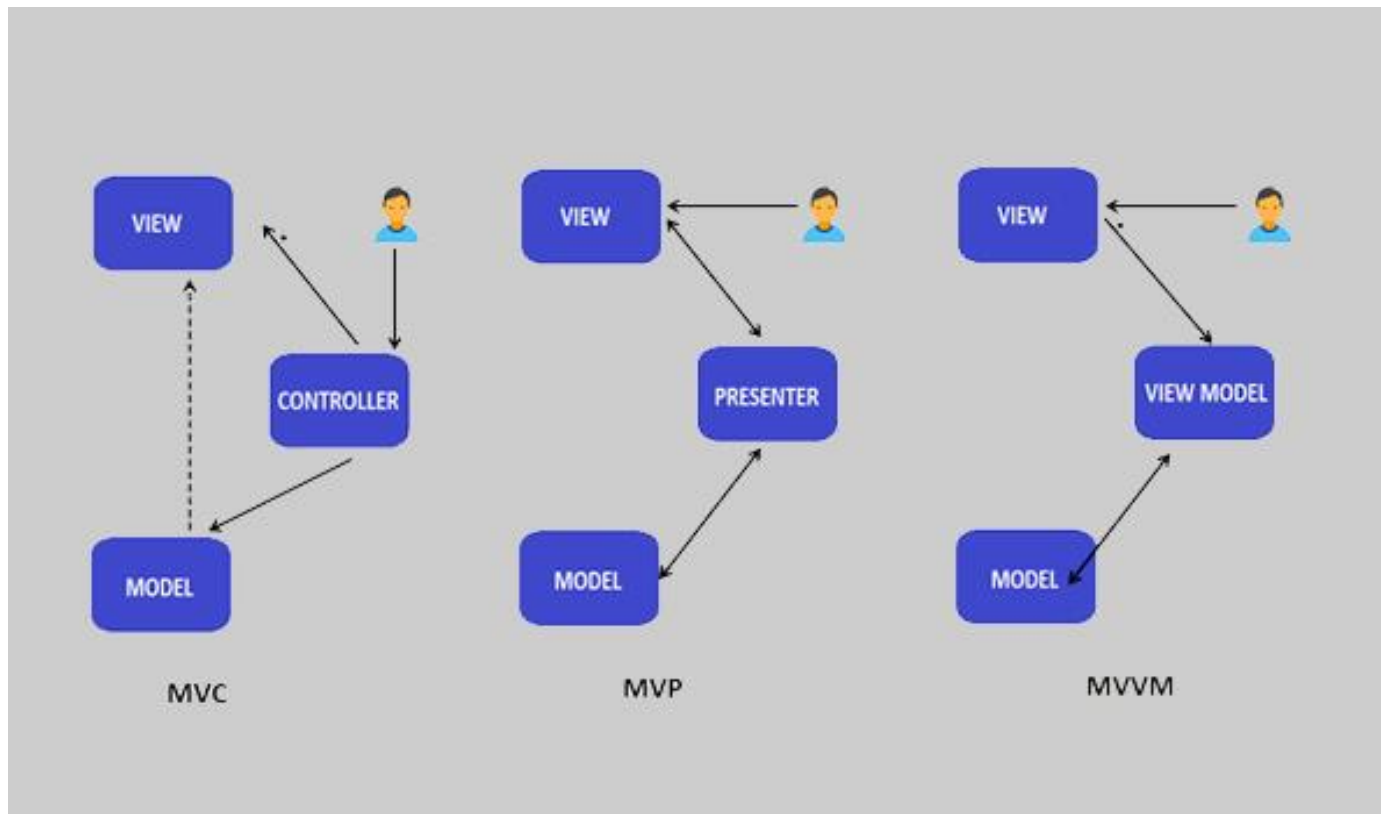
Phân biệt MVC và MVT. Đường dẫn:

- <https://www.geeksforgeeks.org/difference-between-mvc-and-mvt-design-patterns/>
- <https://www.geekinsta.com/difference-between-mvc-and-mvt/>

Tham khảo thêm

Xem thêm các loại mô hình phát triển web khác

- Đường dẫn: <https://wiki.matbao.net/mvc-mvp-mvvm-la-gi-thong-tin-can-biet-ve-cac-mo-hinh-lap-trinh/>



Giới thiệu

- Django là một Web Framework phổ biến nhất được viết bằng Python.
- Hỗ trợ triển khai web nhanh chóng.
- Django sử dụng mô hình MVT (Model-View-Template) tương tự mô hình MVC (Model-View-Controller)
 - V (View) $\Leftrightarrow$ C (Controller)
 - T (Template) $\Leftrightarrow$ V (View)



Tại sao dùng django?

Vì

- Mã nguồn mở và phổ biến nhất.
- Dễ bảo trì: do áp dụng mô hình MVT tương tự mô hình MVC giúp tách biệt các thành phần trong ứng dụng.
- Khả năng mở rộng cao: áp dụng thêm các thư viện hỗ trợ.
- Tích hợp sẵn nhiều chức năng: quản lý trang admin, xác thực và phân quyền, quản lý người dùng...
- ...

Tại sao dùng django?

Các ứng dụng lớn đã phát triển bằng django

- Instagram
- Spotify
- Youtube
- Dropbox
- Mozilla
- Disqus
- ...

Một vài hạn chế

- Cấu trúc phức tạp => cần phải thiết lập nhiều chức năng.
- Không nên sử dụng django để xây dựng những trang web nhỏ.
Có thể dùng Flask Framework xây dựng ứng dụng web nhỏ.
- Tốc độ chậm hơn so với các framework khác.
- Không phù hợp với các ứng dụng real-time.
- Không phải là framework tốt nhất.
- ...

Phương pháp

Trong cmd gõ lệnh để cài django vào trình thông dịch

- `pip install django`

Kiểm tra phiên bản

- `python -m django --version`
- Hoặc, `pip list`

Sử dụng

- `import django`
- `django.get_version()`

Phương pháp

Cài đặt chỉ định phiên bản

- `pip install Django==4.1.7`
- `pip install --pre django`
- `git clone https://github.com/django/django.git`

Tham khảo: <https://www.djangoproject.com/download/>

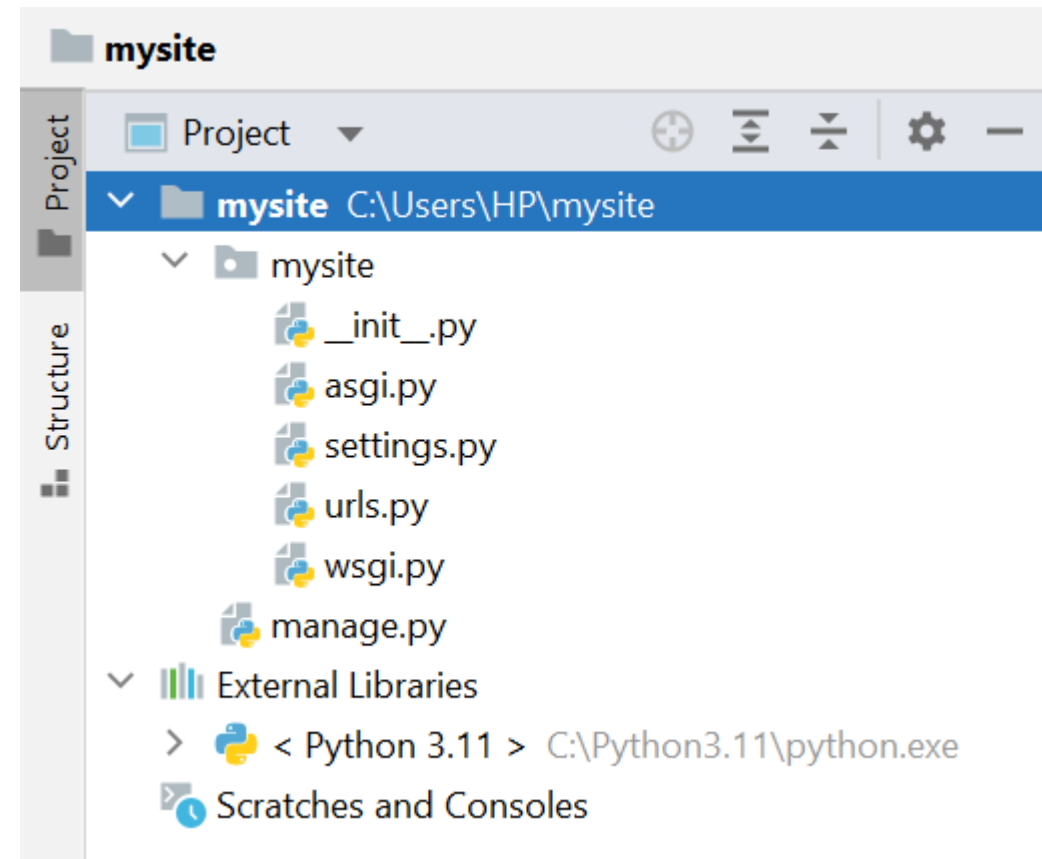
Tạo mới project với Django

Cú pháp tạo project

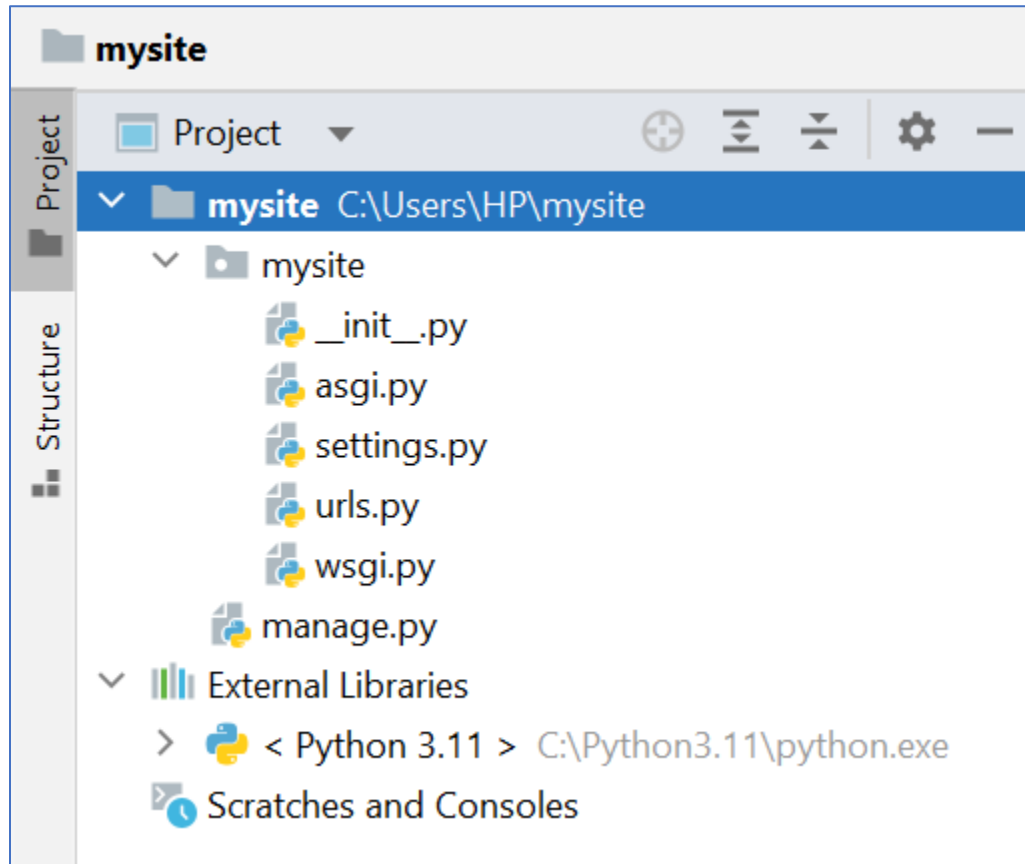
`django-admin startproject <tên project>`

Ví dụ:

- `django-admin startproject mysite`



Tạo mới project với Django

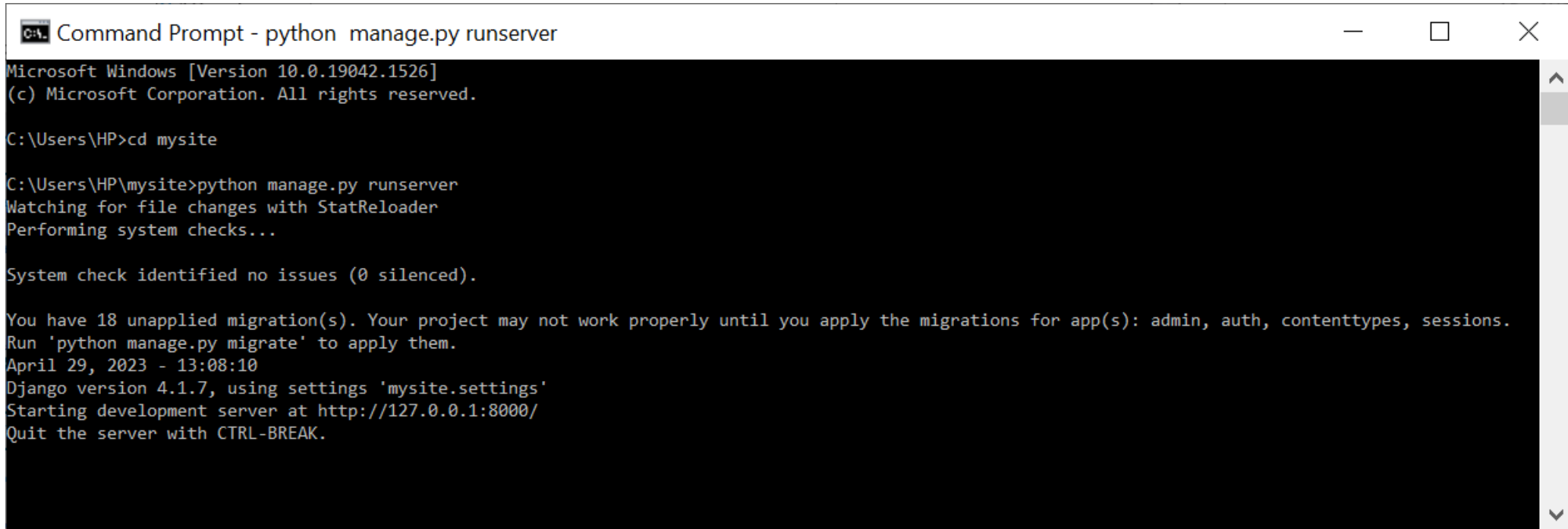


Các thành phần trong project

- **manage.py**: Cho phép LTV tương tác với project theo nhiều cách khác nhau.
- **__init__.py**: Quy định một package trong python.
- **settings.py**: Cấu hình
- **urls.py**: Chứa tất cả khai báo URL trong project và mục lục của trang web.
- **wsgi.py**: Lối vào cho các máy chủ web tương thích WSGI và deploy với WSGI.

Chạy sever

- `cd mysite`
- `python manage.py runserver`



```
Command Prompt - python manage.py runserver
Microsoft Windows [Version 10.0.19042.1526]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>cd mysite

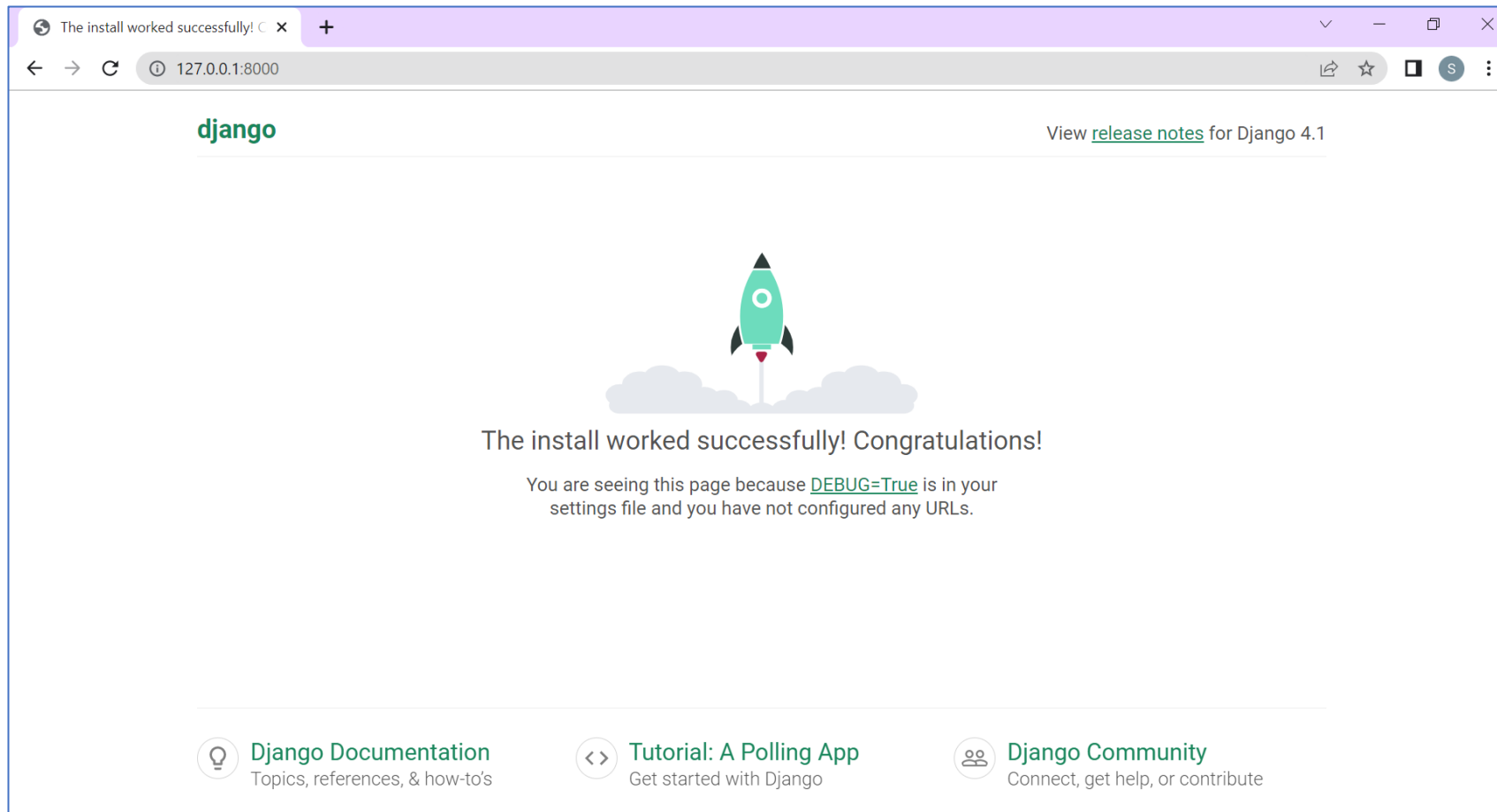
C:\Users\HP\mysite>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
April 29, 2023 - 13:08:10
Django version 4.1.7, using settings 'mysite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

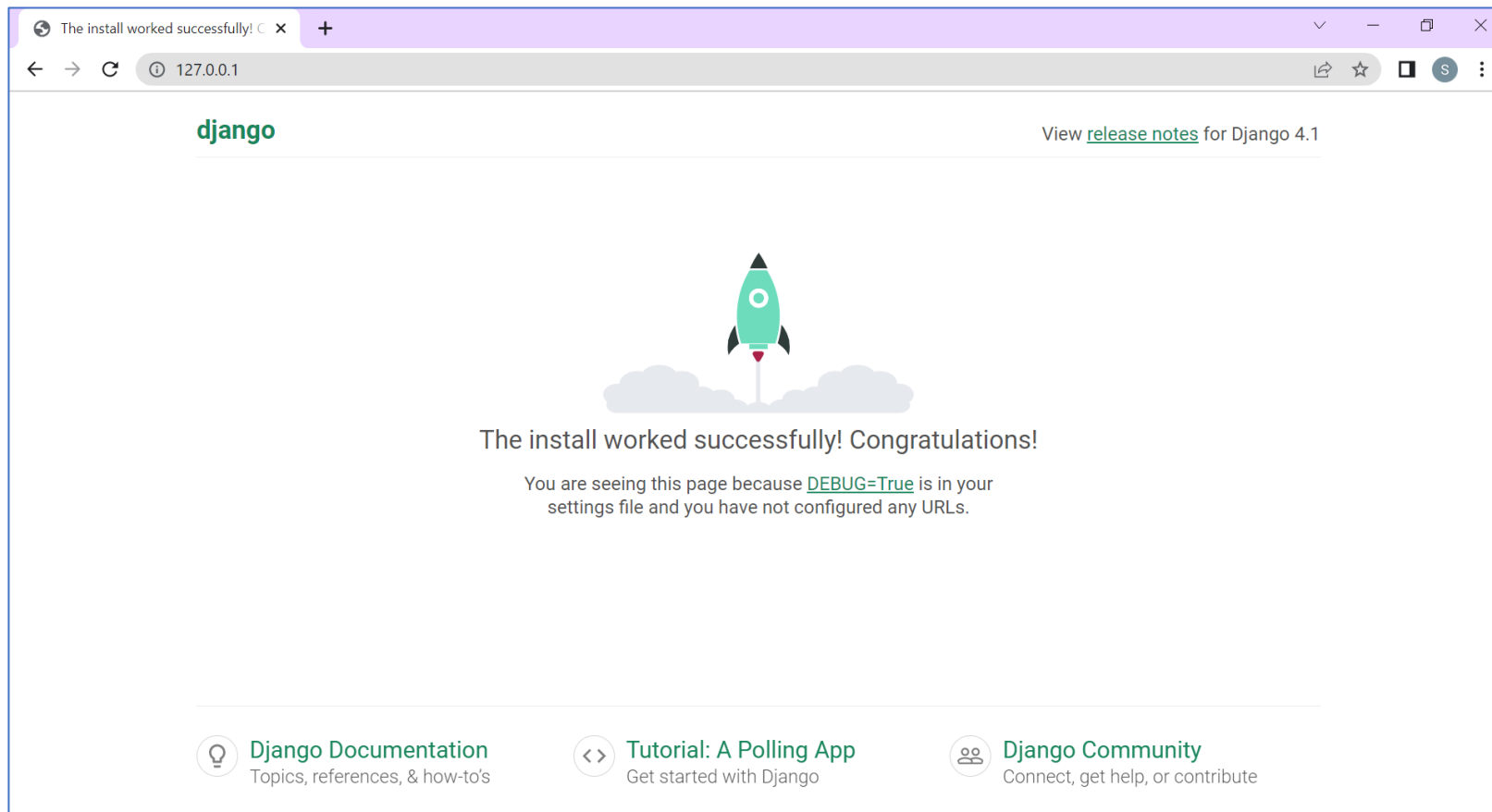
Mở server

- <http://127.0.0.1:8000>



Đổi port

- `python manage.py runserver 80`
- <http://127.0.0.1/>



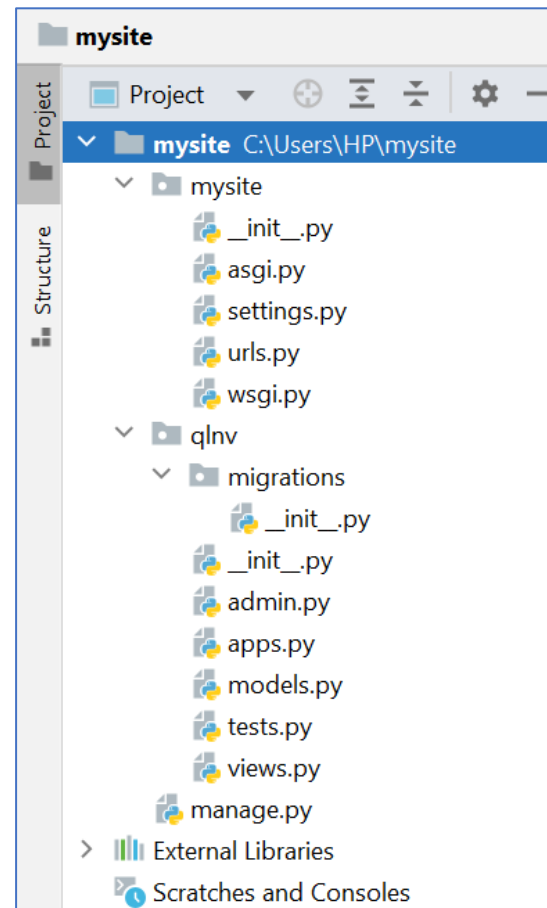
Tạo app

Cú pháp tạo app

`python manage.py startapp <tên app>`

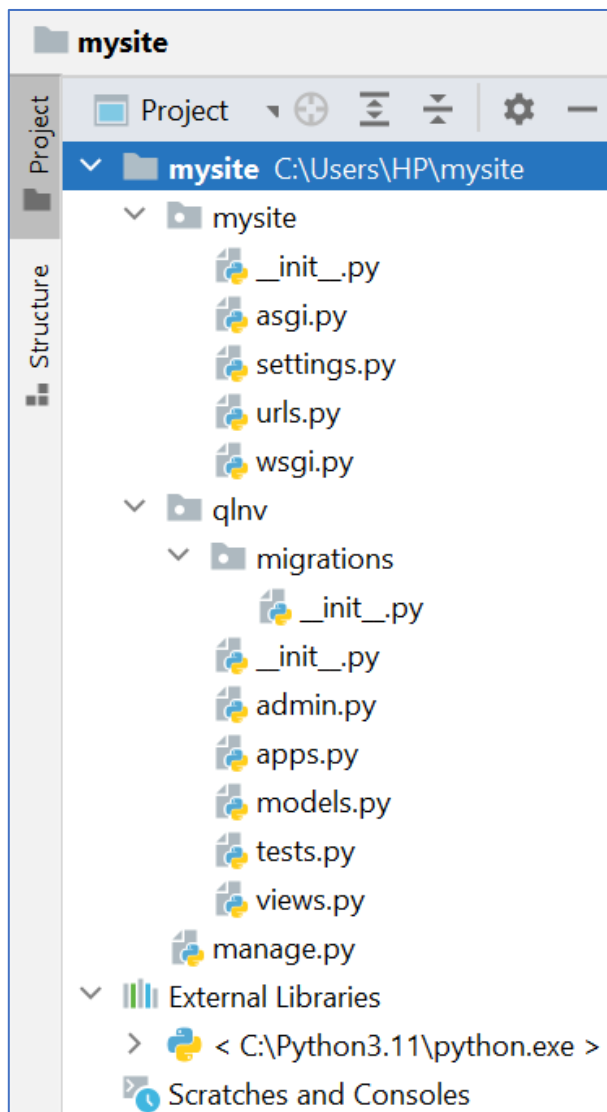
Ví dụ

- `python manage.py startapp qInV`
- Trước khi tạo app nhớ chọn thư mục project: `cd mysite`



App là chức năng của web, các ứng dụng nhỏ để dễ dàng quản lý.

Các thành phần trong app



- **admin.py**: Quản lý đối tượng ứng dụng (thường là mô hình cơ sở dữ liệu) trên trang admin của django
- **apps.py**: Cấu hình các ứng dụng (app)
- **migrations**: 1 package dùng để lưu lại các lịch sử thay đổi của database khi chúng ta thực chỉnh sửa.
- **models.py**: Chứa khai báo các mô hình dữ liệu (model)
- **test.py**: Chứa các đoạn code dùng để kiểm tra của app
- **views.py**: Chức năng như là 1 Controller trong MVC, nhận các yêu cầu, xử lý yêu cầu lấy dữ liệu từ CSDL sau đó trả về template (HTML,CSS,JS)

Sự khác biệt giữa project và app

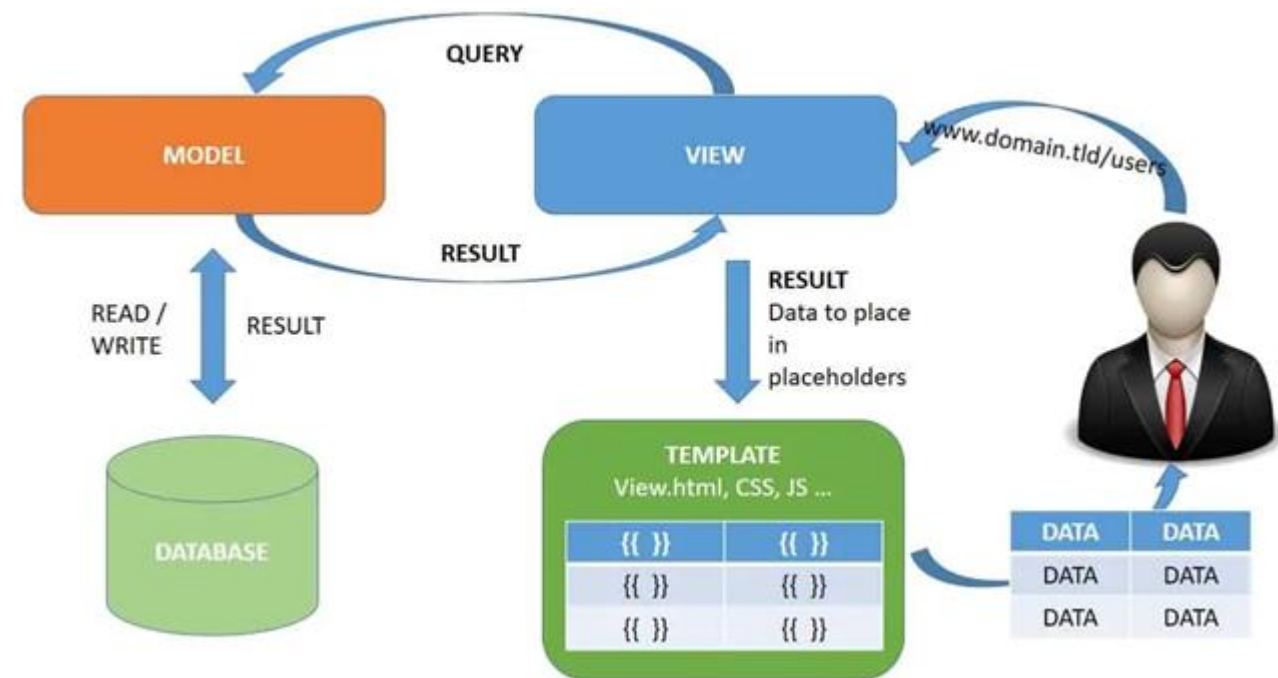
Projects vs. apps

Projects	Apps
• Project là một bộ tập hợp cấu hình và ứng dụng cho một hệ thống web. • Một project có thể chứa nhiều ứng dụng.	• Ứng dụng là một ứng dụng web thực hiện một số chức năng. • Ví dụ: ứng dụng khảo sát, ứng dụng quản lý bán hàng... • Ứng dụng có thể nằm trong nhiều project.

Model – View – Template (MVT)

Giới thiệu

- MVT là một dạng mở rộng của mô hình MVC.
- Bản chất mô hình MVT và MVC đều giống nhau. Gồm 3 thành phần:
 - (1) Xử lý; (2) Phần hiển thị giao diện; (3) Phần thao tác với CSDL.
- Model – **M**: Thao tác CSDL
- View – **V**: Xử lý yêu cầu.
- Template – **T**: Hiển thị giao diện.



Model – View – Template (MVT)

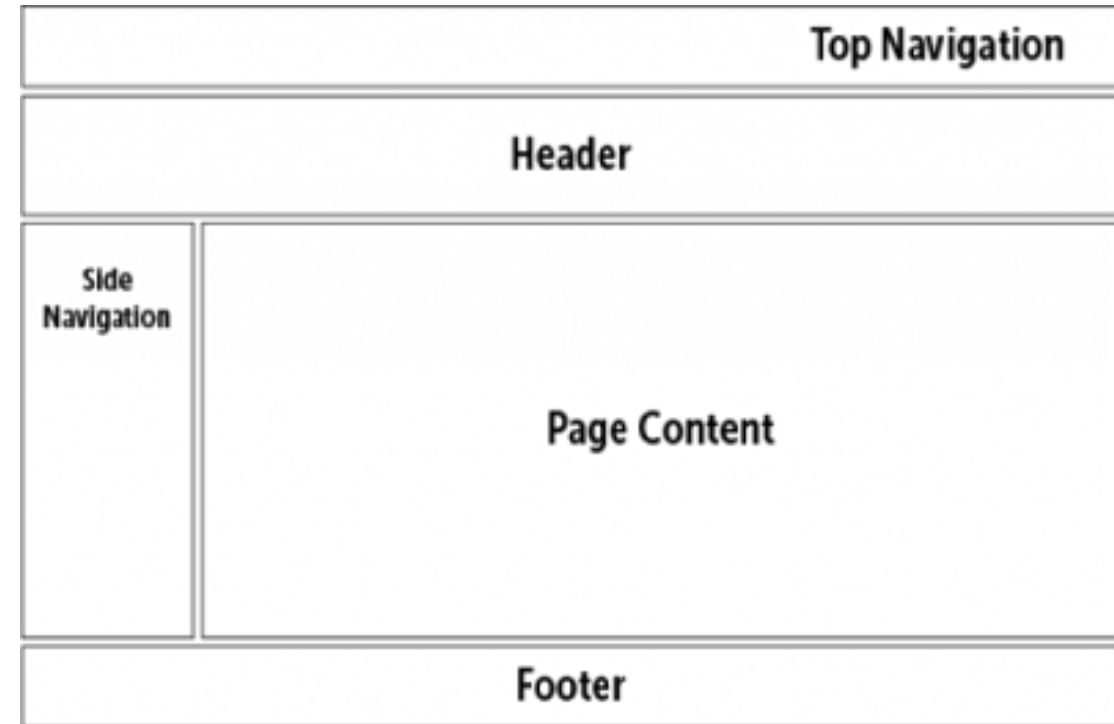
Kiến trúc

- Model – **M**: Thao tác CSDL như MySql, Postgres... để quản lý dữ liệu, tạo các bảng, trường dữ liệu, các ràng buộc. Đơn giản các tác vụ, sắp xếp các bảng thành các model.
- View – **V**: Xử lý yêu cầu để thể hiện lên giao diện người dùng, kết xuất (render) website => Template. Thể hiện bằng HTML/CSS/Javascript và các file Jinja.
- Template – **T**: Là một bản mẫu, hiển thị giao diện, tạo ra các giao diện web. Một template có 2 phần:
 - Phần cố định (static).
 - Phần được bổ sung sau (dynamic).

=> Template gồm các phần tĩnh của đầu ra HTML mong muốn; và cú pháp đặc biệt mô tả cách thức **nội dung động** sẽ được chèn vào.

Giới thiệu

- Là một bản mẫu, hiển thị giao diện, tạo ra các giao diện web.
- Template có 2 phần:
 - Phần cố định (static).
 - Phần được bổ sung sau (dynamic).
- Công cụ hỗ trợ tạo template: Jinja và DTL (Django Templage Language).
- Template trong MVT đóng vai trò như View trong MVC.



DTL: <https://docs.djangoproject.com/en/4.2/ref/templates/language/>

Template – T

- Sẽ có phần riêng (*Phần tiếp theo*) về Template – T: dùng ngôn ngữ Django Template Language – DTL để thiết kế template.

Ví dụ template

```
<!DOCTYPE html>
<html>

<head>
    <title>Minh họa cơ bản</title>
</head>

<body>
    <h1>Xin chào, {{ user_name }}!</h1>
</body>

</html>
```

Ví dụ DTL

```
{% extends "base_generic.html" %}

{% block title %}{{ section.title }}{% endblock %}

{% block content %}
<h1>{{ section.title }}</h1>

{% for story in story_list %}
<h2>
    <a href="{{ story.get_absolute_url }}">
        {{ story.headline|upper }}
    </a>
</h2>
<p>{{ story.tease|truncatewords:"100" }}</p>
{% endfor %}
{% endblock %}
```

So sánh giữa mô hình MVT và MVC

MVT - MVC

Python django - MVT	MVC
Model	Model
Template	View
View	Controller

Ref.: <https://rboflife.home.blog/2021/04/03/django-xay-dung-mo-hinh-mvt-voi-django/>

Giới thiệu

- View - V là thành phần xử lý logic của ứng dụng web, nhận yêu cầu từ người dùng để thực hiện xử lý dữ liệu, render template, trả về phản hồi.

Phương pháp

- Xây dựng hàm hoặc lớp.
- Các kiểu view có sẵn như `TemplateView`, `ListView`, `DetailView`...

Áp dụng: Tạo xử lý cho view

- Mở file: **qlnv/views.py** viết code sau:

```
from django.http import HttpResponse  
  
def index(request):  
    return HttpResponse("Xin chào. Bạn đã đến trang QLNV")
```

- Lớp **HttpResponse** sử dụng để tạo ra phản hồi HTTP từ view.

View

URLconf

- Để gọi view đã tạo, cần ánh xạ nó tới một URL => tạo một URLconf.
- Trong app **qlnv** => tạo file **urls.py** => **qlnv/urls.py**

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path("", views.index, name='index'),
]
```

Tên module **views**

Gọi hàm **index** đã định nghĩa trong module **views**

Thiết lập thêm đường dẫn

root URLconf

- Trỏ URLconf gốc vào mô-đun **qinv.urls**.
- Vào **mysite/urls.py**

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('qinv/', include('qinv.urls')),
    path('admin/', admin.site.urls),
]
```

Thiết lập đường dẫn

Hàm `include()` cho phép tham chiếu các URLconf khác

URLconf

- <http://localhost:8000/admin/>
- <http://localhost:8000/qInv/>

path(**route**, **view**, kwargs=None, name=None)

Giải thích

- Hàm path() dùng để định nghĩa **một tuyến đường** (route) trong file urls.py cho ứng dụng đã tạo.
- Hai đối số bắt buộc: **route**, **view**
 - **route**: là chuỗi chứa mẫu đường dẫn (URL pattern) mà Django sử dụng để xác định một yêu cầu được gửi đến.
 - **view**: hàm hoặc lớp (trong mô-đun view) được gọi khi đường dẫn khớp với yêu cầu.
- Hai đối số tùy chọn: **kwargs** và **name**.
 - **kwargs**: các đối số được chuyển đến hàm xử lý hoặc lớp xử lý
 - **name**: đặt tên cho router, cho phép tham chiếu đến nó ở những nơi khác trong project. Ví dụ như trong template hoặc sử dụng hàm reverse() tạo URL động.

`path(route, view, kwargs=None, name=None)`

Giải thích thêm quy tắc xử lý

route

- Khi xử lý yêu cầu, Django bắt đầu từ mẫu đầu tiên trong danh sách **urlpatterns** và đi xuống.
- So sánh URL được yêu cầu với từng mẫu trong danh sách **urlpatterns** cho đến khi tìm thấy mẫu phù hợp.

view

- Khi tìm thấy một mẫu phù hợp.
- Sẽ gọi hàm tương ứng trong mô-đun view được chỉ định với một đối tượng HttpRequest làm đối số đầu tiên.

Database setup

mysite/settings.py

- Mở file **mysite/settings.py** để cài đặt.
- Theo mặc định, cấu hình sử dụng SQLite.
- SQLite được tích hợp sẵn trong django => không cần cài đặt thêm bất kỳ.
- Có thể sử dụng một CSDL khác như PostgreSQL, MySQL...

Database setup

mysite/settings.py

- Nếu sử dụng CSDL khác thì phải cài đặt các ràng buộc CSDL thích hợp.
- Thay đổi giá trị của key 'default' trong DATABASES để phù hợp với cài đặt kết nối CSDL.
- Mặc định sử dụng sqlite:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

Database setup

mysite/settings.py

- 'ENGINE': built-in database backends
 - 'django.db.backends.sqlite3'
 - 'django.db.backends.postgresql'
 - 'django.db.backends.mysql'
 - 'django.db.backends.oracle'
- 'NAME': Tên CSDL. Giá trị là đường dẫn đến CSDL.
- Giá trị mặc định là `BASE_DIR / 'db.sqlite3'` => lưu trữ file trong thư mục project.

CSDL khác

- Nếu không sử dụng SQLite làm CSDL thì phải thêm các cài đặt bổ sung như USER, PASSWORD, và HOST.

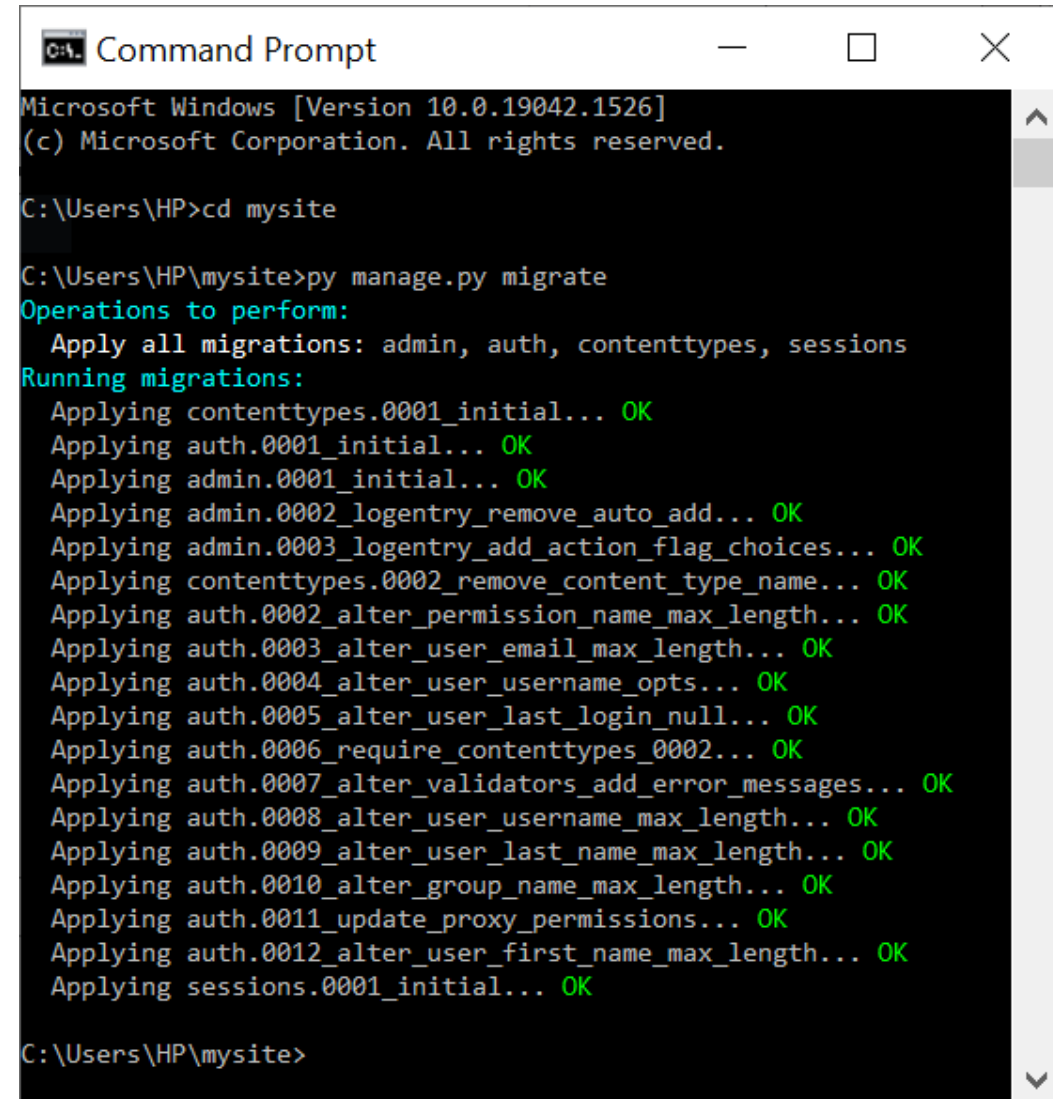
```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'mydatabase',  
        'USER': 'mydatabaseuser',  
        'PASSWORD': 'mypassword',  
        'HOST': '127.0.0.1',  
        'PORT': '5432',  
    }  
}
```

Database setup

Sử dụng CSDL

- `py manage.py migrate`

=> áp dụng các thay đổi mô hình dữ liệu vào CSDL.



```
Command Prompt
Microsoft Windows [Version 10.0.19042.1526]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>cd mysite

C:\Users\HP\mysite>py manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK

C:\Users\HP\mysite>
```

Giới thiệu

- Model dùng để định nghĩa cấu trúc và quy tắc ràng buộc liên quan đến dữ liệu của ứng dụng.
- Mỗi model đại diện cho một bảng trong CSDL.
- Trường (fields) trong model tương ứng với cột của bảng trong CSDL.
- Áp dụng nguyên tắc DRY - Don't repeat yourself: khuyến khích việc tránh lặp lại code và duy trì tính tái sử dụng của code => tránh trùng lặp code.

Phương pháp

- Tạo class kế thừa từ **django.db.models.Model**
 - Khai báo các thuộc tính được khởi tạo từ các trường (field) trong django như: **CharField**, **IntegerField**, **DateTimeField**...
 - Tạo các quan hệ giữa các bảng trong CSDL như quan hệ
 - một-một
 - một-nhiều
 - nhiều-nhiều
- bằng cách dùng trường biểu diễn quan hệ như:
- **OneToOneField**
 - **ForeignKey**
 - **ManyToManyField**

Áp dụng

- **Lớp Model** dùng để tạo model và truy vấn CSDL.
- Trong ứng dụng **qlnv** tạo 1 model tên NhanVien
 - **NhanVien** có các trường dữ liệu:
 - Mã số nhân viên => Kiểu số nguyên
 - Họ tên => Kiểu chuỗi
 - Ngày sinh => Kiểu ngày
 - Giới tính => Kiểu chuỗi
 - Địa chỉ => Kiểu chuỗi

qlnv/models.py

```
from django.db import models

# Create your models here.
class NhanVien(models.Model):
    ma_nv = models.IntegerField()
    ho_ten = models.CharField(max_length=200)
    ngay_sinh = models.DateField()
    gioi_tinh = models.CharField(max_length=10)
    dia_chi = models.CharField(max_length=500)

    def __str__(self):
        return str([self.ma_nv, self.ho_ten])
```

Giải thích:

- Khi định nghĩa một class trong Django models.
- Dù thuộc tính được định nghĩa ở cấp độ class, nhưng mỗi đối tượng (instance) của class sẽ lưu dữ liệu riêng của obj cho các thuộc tính.

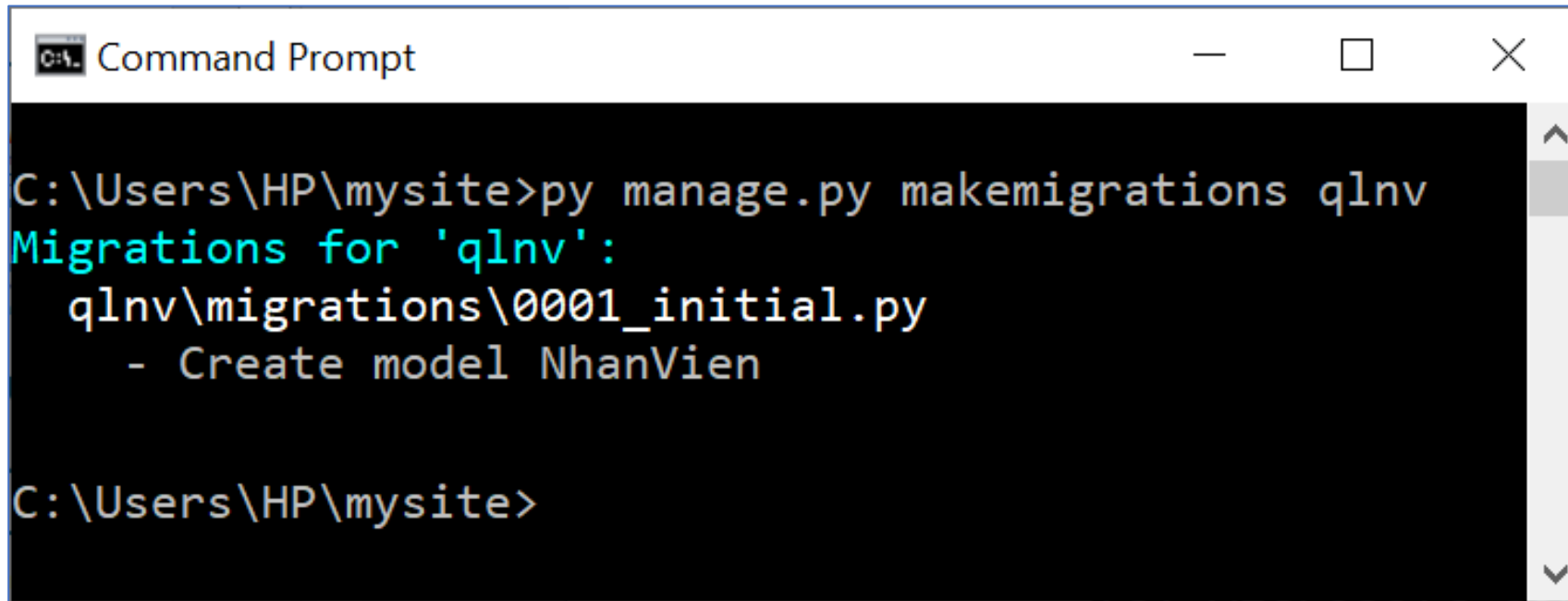
Khai báo app

- Để đưa app vào project thì thêm một tham chiếu tại **INSTALLED_APPS**.
- Lớp **QInvConfig** trong **qInv/apps.py** => đường dẫn là **'qInv.apps.QInvConfig'**
- File: **mysite/settings.py**

```
INSTALLED_APPS = [  
    'qInv.apps.QInvConfig',  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
]
```

Chấp nhận các thay đổi

- Tạo migration cho các thay đổi: `py manage.py makemigrations qlnv`



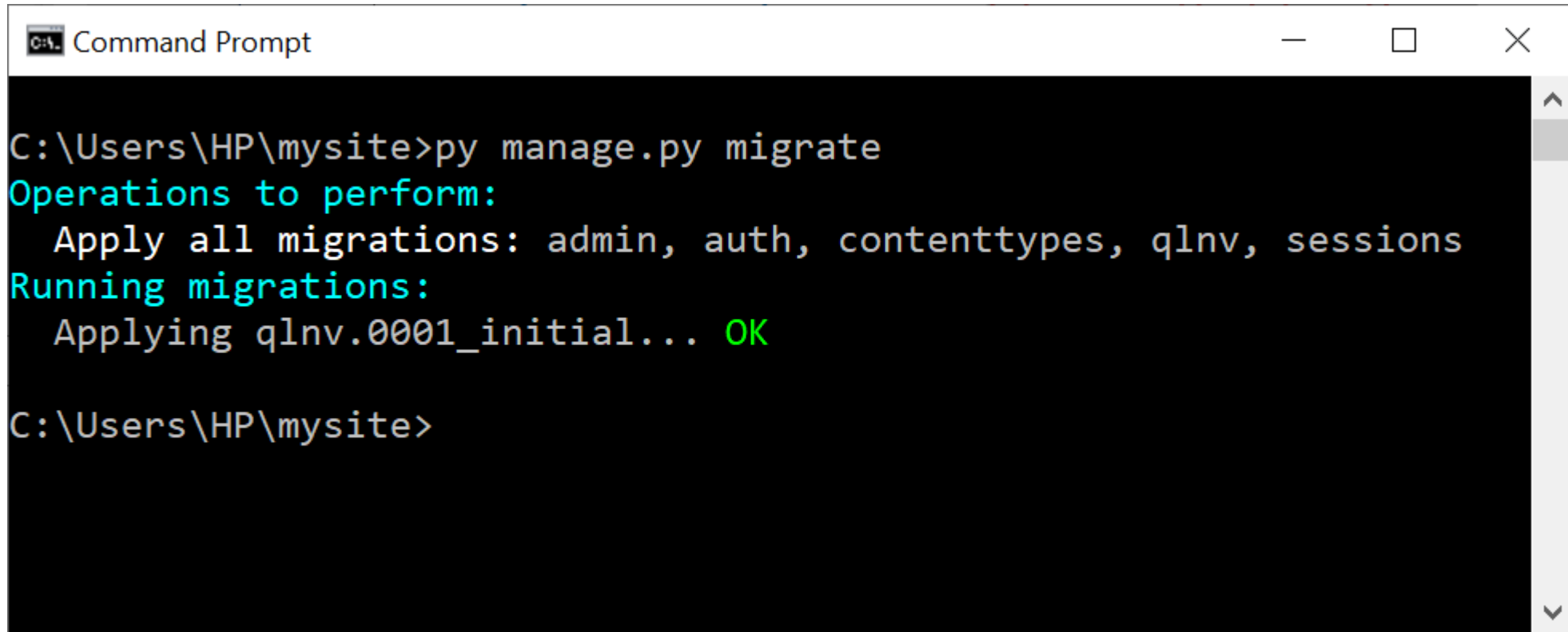
```
Command Prompt

C:\Users\HP\mysite>py manage.py makemigrations qlnv
Migrations for 'qlnv':
  qlnv\migrations\0001_initial.py
    - Create model NhanVien

C:\Users\HP\mysite>
```


Chấp nhận thay đổi

- Chấp nhận thay đổi csdl: `py manage.py migrate`



```
Command Prompt

C:\Users\HP\mysite>py manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, qlnv, sessions
Running migrations:
  Applying qlnv.0001_initial... OK

C:\Users\HP\mysite>
```

Nguyên lý

- Django hoàn toàn tự động hóa việc tạo giao diện quản trị cho các model.
- Tách biệt rõ ràng giữa trang quản lý và trang sử dụng.
- Trang quản trị (trang admin) cho phép thêm nội dung (VD: bài viết mới...) và nội dung đó được hiển thị trên trang sử dụng.
- Django tạo giao diện thống nhất để admin chỉnh sửa nội dung.
- Admin không dành cho khách truy cập trang web, chỉ dành cho người quản lý trang web.

Django Admin

Tạo admin user

- `py manage.py createsuperuser`

```
Command Prompt

C:\Users\HP\mysite>py manage.py createsuperuser
Username (leave blank to use 'hp'): admin
Email address: admin@example.com
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.

C:\Users\HP\mysite>
```

```
Select Command Prompt

C:\Users\HP\mysite>py manage.py createsuperuser
Username (leave blank to use 'hp'):
Email address: admin@abc.com
Password:
Password (again):
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.

C:\Users\HP\mysite>
```

Django Admin

Truy cập

- <http://127.0.0.1:8000/admin>

Django administration

Username:

Password:

Log in

Django administration

WELCOME, **ADMIN**. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups

[+ Add](#) [Change](#)

Users

[+ Add](#) [Change](#)

Recent Actions

My Actions

None available

Django Admin

Thêm app **qlnv** vào trang admin

- qlnv/admin.py =>

```
from django.contrib import admin
from .models import NhanVien

admin.site.register(NhanVien)
```

Site administration

AUTHENTICATION AND AUTHORIZATION	
Groups	+ Add Change
Users	+ Add Change
QLNV	
Nhan viens	+ Add Change

Recent actions

My actions

None available

Tóm tắt tạo ứng dụng cơ bản

- `django-admin startproject mysite`
- `cd mysite`
- `python manage.py runserver` <http://127.0.0.1:8000/>
- `cd mysite`
- `py manage.py startapp qInv`
- `polls/views.py` => Viết code [1]
- Tạo URLconf trong polls directory, tạo file `urls.py` => Viết code [2]
- Trong `mysite/urls.py` ==> code [3]
<http://localhost:8000/admin/>
<http://localhost:8000/qInv/>

Tóm tắt tạo ứng dụng cơ bản

- `python manage.py migrate` => CSDL
- Tạo model `qInv/models.py` => code 4
- Cấu hình lại `mysite/settings.py` thêm tên app tại `INSTALLED_APPS` bằng đường dẫn dấu chấm `'qInv.apps.PollsConfig'`, => code 5
- Thay đổi: `py manage.py makemigrations qInv` <http://127.0.0.1:8000/>
- Xem: `py manage.py sqlmigrate qInv 0001`
- OK: `py manage.py migrate`

- `qInv/models.py` => Cập nhật thêm => code 6 <http://localhost:8000/admin/>
- `qInv/models.py` => Cập nhật thêm => code 7 <http://localhost:8000/qInv/>

- `py manage.py createsuperuser` => tạo admin => xem trang admin
- Thêm app qInv vào trang admin `qInv/admin.py` => code 8

Cơ bản

- Về cơ bản đã thiết lập được web app dùng Django.
- Tạo được Model, View, Template cơ bản.

Tiếp theo...

- Tạo thêm các trang trong ứng dụng `qinv`

Mục tiêu

Trong app qlnv tiếp theo, đặt mục tiêu xây dựng các thành phần:

- Trang “**index**”: hiển thị các nhân viên.
- Trang “**detail**”: hiển thị chi tiết nhân viên.

Nguyên lý

- Trong Django, các trang web và nội dung trang được cung cấp bởi view.
- Mỗi view được trình bày bởi:
 - một hàm,
 - hoặc phương thức,
 - hoặc lớp - class-based view.
- Django sẽ chọn một view bằng cách kiểm tra URL được request (để cho chính xác, phần của URL sau tên miền).
- Để lấy một URL đến một view, Django sử dụng 'URLconfs'. Một URLconf ánh xạ các mẫu URL đến các view.
- Ví dụ URL pattern: /qlnv/<manv>/<details>/

qlnv/views.py

- Tại trang qlnv hiển thị trả về tất cả các nhân viên

```
from django.http import HttpResponse
from .models import NhanVien

def index(request):          # Định nghĩa lại hàm
    #return HttpResponse("Xin chào. Bạn đã đến trang QLNV")
    ds_nv = NhanVien.objects.all()
    output = "</br>".join([nv.ho_ten for nv in ds_nv])
    return HttpResponse(output)
```

qlnv/views.py

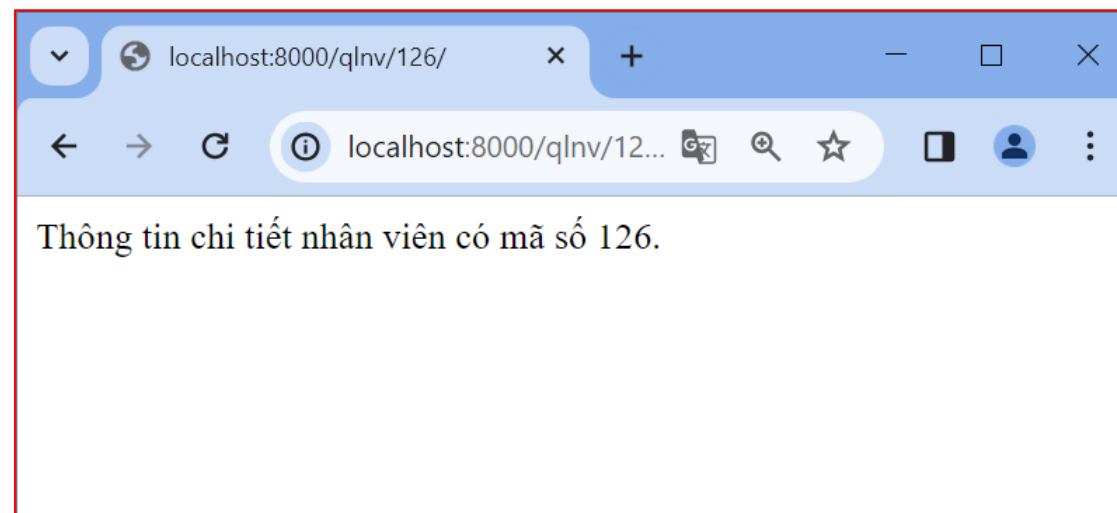
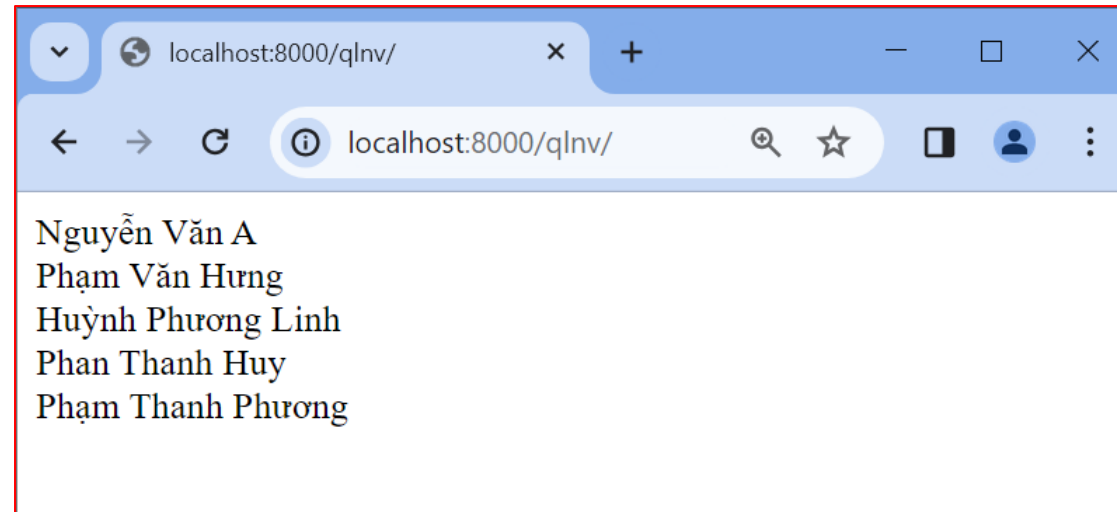
```
def detail(request, manv):  
    return HttpResponse("Thông tin chi tiết nhân viên %s." % manv)
```

qInv.urls

- Sau khi cài đặt xong view thì cài đặt liền `qInv.urls`

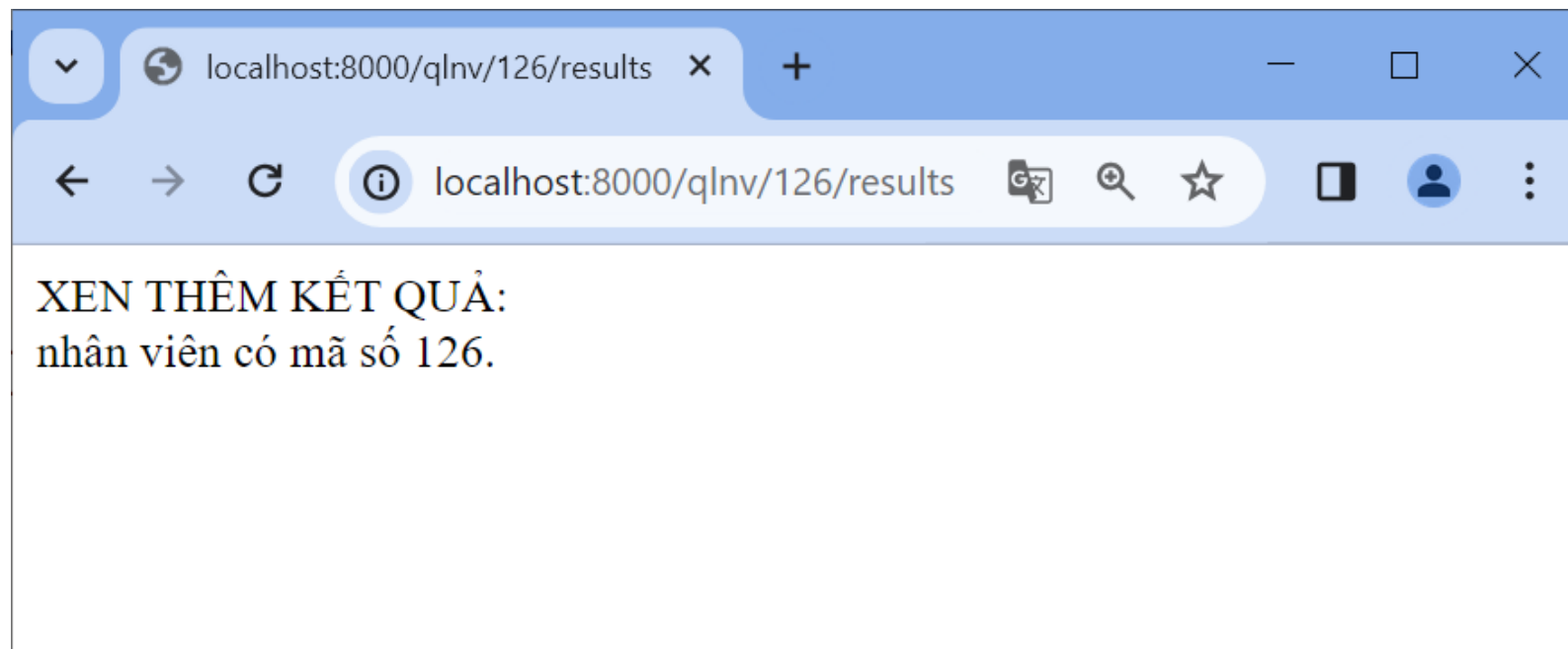
```
urlpatterns = [  
    # Vd: /qInv/  
    path("", views.index, name="index"),  
  
    # Vd: /qInv/122/  
    path("<int:manv>/", views.detail, name="detail"),  
  
    # Vd: /qInv/122/results/ => Bài tập  
    path("<int:manv>/results/", views.results, name="results"),  
]
```

Kết quả



qInv/views.py

```
def results(request, manv):  
    return HttpResponse("XEN THÊM KẾT QUẢ:<br>nhân viên có mã số %s." % manv)
```



- `django-admin startproject demo_viz`
- `cd demo_viz`
- `python manage.py runserver`

- `cd demo_viz`
- `py manage.py startapp demo_matplotlib`

- `py manage.py migrate`
- `py manage.py makemigrations demo_matplotlib`
- `py manage.py sqlmigrate demo_matplotlib 0001`
- `py manage.py migrate`



Thank you

Hỏi - Đáp

